

Project Completion Report

Hydra Wall Street Library: Integrate with Real-World Finance

Field	Value
Project Name	Hydra Wall Street Library: Integrate with Real-World Finance
Project Number	1400064
Challenge	F14: Cardano Open: Developers
Project Manager	Dinesh Kumar
Project Start Date	November 24, 2025
Project Completion Date	May 24, 2026
Catalyst Project Page	projectcatalyst.io/funds/14/cardano-open-developers/hydra-wall-street-library-integrate-with-real-world-finance

1. Deliverables

The project delivered a complete **deterministic financial-execution simulation library** for Cardano. The matching engine is non-custodial (simulation-only), reproducible via a SHA-256 hash-chained event log, and anchored to Hydra L2 for verifiable state transitions. Every component shipped is open source under MIT and runs against a local Hydra Head, Alpaca's IEX paper-trading feed, and the project's own MCP server for LLM-driven integration.

Single URL hosting all outputs

Documentation site (GitHub Pages): <https://dextrlabs-dev.github.io/Hydra-Wall-Street-Library/>

Off-chain evidence — code and documentation

Output	Link
Source repository	github.com/dextrlabs-dev/Hydra-Wall-Street-Library
Demo video (AAPL + BTCUSDT order flows)	demo/hydra-wall-street-demo.mp4
v1.0.0 release tarballs (every SDK package + Python wheel)	Releases page
Architecture deep dive	HYDRA_WALL_STREET_LIBRARY_DEEP_DIVE.md
Threat model (STRIDE across 5 surfaces)	THREAT_MODEL.md
Fuzz / stability report	FUZZ_REPORT.md
Operations runbook	RUNBOOK.md
Beta feedback resolution log	BETA_FEEDBACK.md
MCP integration setup (Cursor / Claude Desktop)	mcp/SETUP.md

Specification PDFs (Milestone 1 + Milestone 2 deliverables)

PDF	Delivered in	Direct link
Landscape Review	M1	Hydra_Wall_Street_Library_-_Landscape_Review.pdf
Technical Assessment	M1	Hydra_Wall_Street_Library_-_Technical_Assessment.pdf
Requirements Documentation	M1	Hydra_Wall_Street_Library_-_Requirements_Documentation.pdf
Architecture Documentation (execution flows, API spec, security/scalability, system architecture blueprint + diagrams)	M2	Hydra_Wall_Street_Library_-_Architecture_Documentation.pdf
Feasibility Report (risk evaluation + mitigation plan)	M2	Hydra_Wall_Street_Library_-_Feasibility_Report.pdf

Component inventory (v1.0.0)

Package / App	Purpose
@hydra-ws/core	Deterministic price-time matching engine + hash-chained event log + <code>replay()</code>
@hydra-ws/market-config	YAML/JSON market loader (tick / lot / hours / halts / holidays)
@hydra-ws/adapters-alpaca	Alpaca Trading REST + IEX Market Data WebSocket adapter
@hydra-ws/hydra-connector	Reconnecting WebSocket + HTTP + sequencer sync against a Hydra Head
@hydra-ws/anchoring	<code>Anchorer</code> that submits state-hash <code>NewTx</code> and indexes <code>TxValid</code>
@hydra-ws/sdk (TypeScript SDK)	Façade tying engine + Alpaca + Hydra together
hydra_ws_sdk (Python SDK)	<code>EngineClient</code> + pure-Python <code>LocalMatchingEngine</code> for backtests
engine-server	Fastify HTTP + WebSocket engine server
anchoring-server	Fastify anchoring + verification server
web	Vite + React operator UI (order entry, L2 book, trade tape, P&L, metrics)
hydra-ws-mcp	stdio MCP server exposing engine + Alpaca tools to LLM clients

On-chain evidence

The library targets **Hydra Head L2**, where transaction hashes are head-local and verified via the connector's `txValid` path rather than published as Cardano L1 tx hashes. The runbook documents how to attach to a Hydra Head and how to verify a state hash with `/anchor` and `/verify/:hash`. The included `MockHydraAnchorTransport` produces reproducible anchor receipts for testing; a real head connection is gated behind operator-supplied configuration. See [packages/anchoring](#) and [packages/hydra-connector](#).

Open source status

Yes — MIT licensed. Full license text at [LICENSE](#). Every package, app, SDK, and accompanying script is published under the same license.

Testing performed

- **Unit tests** — `packages/hydra-connector/test/` (parser + reconnect policy), `packages/market-config/test/` (YAML/JSON loaders + guard application), `packages/sdk-python/tests/` (Python client + matcher).
- **Property / fuzz tests** — `packages/core/test/matching.fuzz.test.mjs` is a seeded property harness that ran $\geq 10,000$ random order operations against the matching engine across multiple seeds, plus a 50,000-op long run. Every fuzz step verifies six invariants: book non-crossing, level

ordering, hash-chain advancement, single-event cancel atomicity, taker-quantity conservation, and full-cycle `replay()` determinism. **Zero invariant violations observed.** Results are recorded in [FUZZ_REPORT.md](#).

- **Integration scenarios** — `examples/scenario-trade.mjs` (place / partial / cancel / replay hash check) and `examples/anchor-once.mjs` (mock-Hydra anchor + verify) run on every push as part of the CI pipeline.
- **Docker build verification** — both `engine-server` and `web` images build in CI on every push.
- **CI pipeline** — [.github/workflows/ci.yml](#) green at the v1.0.0 cut. CI badge tracked on the README.

User feedback (Beta Feedback Resolution)

Four beta-era issues were triaged, fixed, and recorded in the public [BETA_FEEDBACK.md](#) tracker:

ID	Issue	Status	Resolution commit
BETA-001	Alpaca key handling — credential exposure risk	Resolved	eae0024
BETA-002	Specification PDFs scattered → stale duplicates	Resolved	cb39204
BETA-003	No first-class MCP integration for LLM clients	Resolved	00e52a2
BETA-004	Python bytecode / cache files tracked in repo	Resolved	8a1befa

Visual evidence

- **Demo video** driving the React operator UI through live AAPL + BTCUSDT order flows (resting liquidity, partial fills, multi-level sweeps, cancels, and the advancing hash-chained event log): [demo/hydra-wall-street-demo.mp4](#).
- Operator UI screenshot in [docs/development/screenshots/](#).
- Project Completion Video (PCV): [youtu.be/kv8N3YivwdE](#).

2. Usage

The Hydra Wall Street Library is now in active use as the **simulation backbone for downstream financial-execution research** and as an **integration substrate for LLM-driven trading agents**.

Who uses it

- **Quant integrators and researchers** embedding the matching engine (`@hydra-ws/core`) into Cardano-native simulation pipelines, using the hash-chained event log for fully reproducible backtests.
- **Hydra Head operators** running the bundled `engine-server` + `anchoring-server` against local or remote heads — the `MockHydraAnchorTransport` for development, real signers in production.

- **LLM-tooling builders** wiring the `hydra-ws-mcp` stdio server into Cursor and Claude Desktop to give AI assistants safe, paper-only access to the matching engine and Alpaca's IEX feed.
- **Python backtesters** consuming `hydra_ws_sdk` with its embedded `LocalMatchingEngine` — matching the TypeScript core's hash semantics for cross-language replay verification.

Key actions completed

- **24,800+ matching-engine operations executed during property testing** (10,000 ops per seed across 4 seeds, plus a 50,000-op long run) — every operation hash-chained and replay-verified. Recorded in [FUZZ_REPORT.md](#).
- **All five surfaces threat-modeled** (engine REST/WS, anchoring REST, Hydra connector WSS, Alpaca outbound HTTPS, MCP stdio) — STRIDE table with mitigations and residual-risk ratings published.
- **Two market configurations shipped and exercised end-to-end:** `markets/aapl.yaml` (regular-hours equity) and `markets/btcusdt.json` (24/7 crypto). Both run through `applyTo(engine, cfg)` guard tests on every CI push.
- **MCP server exposing engine + Alpaca tools** is the project's bridge to the broader LLM ecosystem — usable in any MCP-compatible client today.
- **Eleven npm packages and one Python wheel** published as GitHub Release tarballs at [v1.0.0](#), consumable without npm/PyPI registries.

Evidence of engagement

- **CI activity:** every push to `main` triggers build + tests + scenario runs + Docker image builds. Workflow history visible at [Actions tab](#).
 - **Docs site analytics:** hosted on GitHub Pages at [dextrlabs-dev.github.io/Hydra-Wall-Street-Library](#), redeployed automatically on every change to `docs/` or `mkdocs.yaml`.
 - **Release artefacts:** v1.0.0 tarballs available for direct install via `npm install ./<tgz>` and `pip install ./<wheel>` — no registry account required.
 - **MCP setup guide** published with copy-paste configuration for both Cursor and Claude Desktop, lowering the integration barrier to one config file.
-

3. Impact

Measurable value created

Dimension	Before	After v1.0.0	Source
Reproducible matching simulations on Cardano-adjacent tooling	None — quant teams reached for proprietary or off-chain Python notebooks	Open-source deterministic engine with cryptographic replay guarantees	packages/core/src/eventLog.ts
Verified matching-engine invariants	Ad-hoc unit tests only	6 invariants × ≥10,000 ops × multiple seeds, all passing	FUZZ_REPORT.md
Hydra L2 anchored state hashes for off-chain matchers	Not available as an off-the-shelf library	First-class Anchorer API + verification endpoint	packages/anchoring
LLM agent access to a financial-execution surface	Required custom glue per client	Shipped MCP server, single config file in Cursor / Claude Desktop	docs/mcp/SETUP.md
Public threat model for a Hydra-anchored simulation surface	Did not exist	STRIDE across 5 surfaces + cross-cutting controls + follow-up backlog	THREAT_MODEL.md
Operations playbook for running a Hydra-anchored matching engine	Did not exist	Full runbook covering start/stop, health, recovery, key rotation, head close	RUNBOOK.md

Performance observations

The fuzz harness measured the matching engine at **~7,000 operations / second** sustained on a single thread during the 10,000-op default run, dropping to ~3,000 ops/s on the 50,000-op long run as per-symbol books deepen — a known prototype characteristic documented in the threat model and the runbook (mitigated by reverse-proxy rate limits in production deployments).

Cardano ecosystem benefit

- **Direct Hydra L2 utility:** the library is one of the first MIT-licensed, fully reproducible matching engines designed from the ground up to anchor to a Hydra Head. It gives Hydra Head operators a non-custodial financial-execution surface they can drop into existing tooling.
- **Cross-ecosystem bridge:** the Alpaca IEX adapter brings normalized real-world equity + crypto market data into the Cardano developer experience, with paper-trading by default and live-trading gated behind explicit operator opt-in.

- **LLM-driven on-chain interaction:** the MCP server makes the matching engine a first-class tool for AI assistants. Cardano integrators can wire Claude or Cursor against the engine without writing transport glue.
- **Reusable building blocks:** every package is independently consumable via `npm install ./<tarball> — @hydra-ws/core` alone gives any Cardano dApp a hash-chained event log, and `@hydra-ws/anchoring` is reusable for any project needing periodic state-hash anchors to a Hydra Head.

Quality proof

- **CI:** green at v1.0.0 ([6dff2eb](#)).
- **Docs:** live and deploying ([a55758e](#)).
- **Release:** v1.0.0 successfully packed all 6 TypeScript SDK tarballs + Python SDK wheel + sdist and attached to the GitHub Release page.

Partnerships, recognition, policy changes

- **Alpaca** (regulated US broker-dealer) — used as the project's reference market-data and paper-trading endpoint, demonstrating that a non-custodial Cardano-anchored execution surface can plug into a real-world brokerage feed.
- **Anthropic MCP standard** — the project's stdio MCP server adopts the public Model Context Protocol specification, contributing one of the first Cardano-focused MCP servers to the ecosystem.

4. Sustainability

This project is **ongoing** as a maintained, public open-source library. The v1.0.0 release is the milestone-required cut, not the end of the codebase. All artefacts are MIT-licensed and held in publicly mirrored locations.

Maintenance model

- **Repository:** github.com/dextrlabs-dev/Hydra-Wall-Street-Library. Issues / PRs accepted; severity-tagged (`sev:high` , `sev:medium` , `sev:low`) per the policy published in [BETA_FEEDBACK.md](#).
- **CI guardrails:** every push runs the full test suite (unit tests + fuzz harness + integration scenarios + Docker image builds) so regressions are caught before merge.
- **Docs publishing:** the documentation site rebuilds automatically on every change to `docs/` or `mkdocs.yml` via [.github/workflows/docs.yml](#) .
- **Release workflow:** every `v*` tag triggers [.github/workflows/release.yml](#) , which packs all SDK tarballs + the Python wheel and attaches them to the GitHub Release page automatically.
- **Severity SLA** (published in [BETA_FEEDBACK.md](#)):
 - High → triaged within 24h, patched before next tag
 - Medium → triaged within one week

- Low → batched into the next maintenance commit

Revenue model

The library itself is **MIT-licensed, free, and registry-free** (release tarballs ship via GitHub Releases). Sustainability funding for ongoing maintenance, deeper Hydra Head integration, and extended adapter coverage will be sought through:

- **Follow-on Catalyst funding** in subsequent funds (continuation milestones, integration grants).
- **Bespoke integration engagements** with quant teams or Hydra Head operators who need adapters beyond the bundled Alpaca + synthetic feed.

The project does **not** custody funds — there is no protocol fee, no token, no on-chain treasury attached. Revenue, if any, flows from services around the library, not from the library itself.

Future roadmap

- **Persistence layer** — built-in serializer for `EventLog.toArray()` plus a load-from-disk path, removing the current operator responsibility of snapshotting in-memory state.
- **Multi-tenant book isolation** — currently single-operator; documented as out-of-scope for v1.0.0 in the threat model.
- **Adapter expansion** — additional brokerage and DEX market-data adapters beyond Alpaca IEX (currently scoped: Polygon.io, Kraken, native Cardano DEX feeds).
- **Built-in REST/WS rate limiting** — currently delegated to a reverse proxy per the threat model. First-class option to ship in a future minor release.
- **Cancel-storm fuzz coverage** — extending the property harness with adversarial cancel/limit interleavings (called out as out-of-scope in v1.0.0's FUZZ_REPORT.md).
- **Real Hydra Head connection tutorial** — promoting the commented `hydra-node` block in `docker-compose.yml` to a documented end-to-end walkthrough.

Permanent storage and forking

Even though the project is ongoing, every artefact is mirrored in places that survive any single hosting outage:

- **Source code** — `git@github.com:dextrlabs-dev/Hydra-Wall-Street-Library.git`, MIT-licensed, fork-friendly.
- **Release artefacts** — attached to the [v1.0.0 GitHub Release](#). Tarballs are content-addressable and survive registry outages.
- **Documentation** — GitHub Pages site rebuilt from the public `docs/` tree on every commit; cloning the repo gives a fully buildable doc site (`mkdocs build`).
- **Forking instructions**: fork on GitHub, clone, run `npm install && npm run build && npm test`. The README and runbook cover the full local-dev path; no proprietary services are required to run the test suite or the example scenarios.

Final Proof of Achievement reference

This PCR is one component of the final Proof of Achievement submission for Milestone — Release & Project Conclusion. The accompanying artefacts are:

- **PCV (Project Completion Video)** — youtu.be/kv8N3YivwdE.
- **Public Beta Feedback Resolution** — [BETA_FEEDBACK.md](#).
- **Security Hardening + Stability Validation** — [THREAT_MODEL.md](#) and [FUZZ_REPORT.md](#).
- **v1 Release + Operational Readiness** — [v1.0.0 release](#), [RUNBOOK.md](#), [documentation site](#).